

Flutter for Angular Developers

This document is a practical guide for Angular developers who are starting with Flutter in a project that uses Spring Boot as the backend.

1. The big picture

If you already know Angular, think of Flutter as a UI framework where:

- Angular components become Flutter widgets
- Angular templates become widget trees
- Angular services become Dart service classes
- Angular routing becomes Flutter navigation
- Angular forms become Flutter input widgets

The main difference is that Flutter uses Dart and widgets instead of HTML, TypeScript, and Angular decorators.

2. Core mental model

Angular way

- You build UI with components and templates
- You bind data with interpolation and property binding
- You manage state in components or services
- You navigate between pages with the Angular router

Flutter way

- You build UI with widgets
- You compose widgets into a tree
- You manage state inside StatefulWidget or through external state management tools
- You navigate between screens using the Navigator

In short:

- Angular = components + templates + services

- Flutter = widgets + widget tree + service classes

3. Entry point of the app

In Angular, your app usually starts from a root module or bootstrap file.

In Flutter, the app starts from:

- lib/main.dart

Example:

```
void main() {  
  runApp(const MyApp());  
}
```

This is the equivalent of bootstrapping the Angular application.

4. Components vs widgets

Angular component

```
@Component({  
  selector: 'app-home',  
  templateUrl: './home.component.html'  
})  
export class HomeComponent {}
```

Flutter widget

```
class HomePage extends StatelessWidget {  
  const HomePage({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Home')),
```

```
body: const Center(child: Text('Hello')),  
);  
}  
}
```

Key idea

A widget is the basic building block of UI in Flutter.

5. Templates vs widget tree

Angular template

Hello

Save

Flutter widget tree

```
Column(  
  children: [  
    Text('Hello'),  
    ElevatedButton(  
      onPressed: save,  
      child: const Text('Save'),  
    ),  
  ],  
)
```

Important difference

In Flutter, the UI structure is built by nesting widgets. There is no HTML markup.

6. State management

Angular

You often use:

- component state
- services
- RxJS / BehaviorSubject
- NgRx

Flutter

You commonly use:

- StatefulWidget for local state
- Provider for simple shared state
- Riverpod for more scalable state management
- Bloc/Cubit for larger applications

Example of local state in Flutter

```
class CounterPage extends StatefulWidget {  
  const CounterPage({super.key});  
  
  @override  
  State createState() => _CounterPageState();  
}  
  
class _CounterPageState extends State {  
  int count = 0;  
  
  void increment() {  
    setState(() {  
      count++;  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Center(child: Text('$count')),  
    );  
  }  
}
```

```
floatingActionButton: FloatingActionButton(
  onPressed: increment,
  child: const Icon(Icons.add),
),
);
}
}
```

This is similar to component-level state in Angular.

7. Directives and loops

Angular

Welcome

```
{{ item }}
```

Flutter

```
if (isLoggedIn) const Text('Welcome');
```

```
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    return ListTile(title: Text(items[index]));
  },
);
```

Key idea

Flutter uses normal Dart control flow and widget composition instead of Angular-style structural directives.

8. Services and API calls

In Angular, services are often used to call backend APIs.

In Flutter, you create Dart classes to do the same thing.

Example service class

```
class NoteService {  
  Future<>> fetchNotes() async {  
    final response = await http.get(Uri.parse('http://localhost:8080/api/notes'));  
  
    if (response.statusCode == 200) {  
      return jsonDecode(response.body) as List<>;  
    }  
  
    throw Exception('Failed to load notes');  
  }  
}
```

This is the Flutter equivalent of an Angular service.

9. Working with Spring Boot backend

If the backend is built using Spring Boot, Flutter communicates with it over HTTP/REST.

Typical backend endpoints

- GET /api/notes
- POST /api/notes
- PUT /api/notes/{id}
- DELETE /api/notes/{id}

Flow

1. Flutter UI requests data from Spring Boot
2. Spring Boot returns JSON
3. Flutter parses the JSON into Dart objects
4. Flutter updates the UI

Example architecture

- Flutter screen displays a list of notes
- NoteService calls the REST API
- The screen uses the returned data to render UI

This is very similar to Angular calling a backend service from a component.

10. Routing and navigation

Angular routing

- routerLink
- RouterModule
- ActivatedRoute

Flutter navigation

```
Navigator.push(  
  context,  
  MaterialPageRoute(builder: (context) => const DetailPage()),  
);
```

And to go back:

```
Navigator.pop(context);
```

Key idea

Flutter navigation is explicit and widget-based.

11. Forms

Angular form

- template-driven forms

- reactive forms

Flutter form

```
TextField(  
  controller: _controller,  
  decoration: const InputDecoration(labelText: 'Title'),  
);
```

You can also use:

- Form
- TextEditingController
- DropdownButton
- Checkbox
- Radio

12. Styling and theming

Angular

You use CSS and Angular component styles.

Flutter

You use widgets and theme data.

Example:

```
ThemeData(  
  colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),  
)
```

And widget styling:

```
Container(  
  padding: const EdgeInsets.all(16),  
  child: const Text('Styled box'),  
)
```

13. Folder structure recommendation

A Flutter project for a Spring Boot-based app can be organized like this:

```
lib/  
main.dart  
screens/  
widgets/  
models/  
services/  
providers/
```

Suggested purpose

- screens: full pages/screens
- widgets: reusable UI pieces
- models: Dart classes for API data
- services: HTTP requests and backend communication
- providers: shared state

This structure is similar to Angular modules, components, and services.

14. A practical Angular-to-Flutter cheat sheet

```
| Angular | Flutter |  
|---|---|  
| Component | Widget |  
| Template | Widget tree |  
| @Component | StatelessWidget / StatefulWidget |  
| *ngIf | if statement inside build |  
| *ngFor | ListView.builder or for loop |  
| Service | Dart class |  
| HttpClient | http package |  
| Router | Navigator |
```

| FormControl | TextEditingController |
| CSS | Container, Padding, ThemeData |

15. Best practices for this project

1. Keep UI screens separate from business logic
2. Use service classes for backend communication
3. Create models for JSON responses
4. Use state management for shared data
5. Follow a consistent folder structure
6. Keep the app modular so it is easy to scale

16. Example workflow for this project

If you are building a notes app with Spring Boot backend:

1. Create a screen for displaying notes
2. Create a service that calls the backend API
3. Parse JSON into a Dart model
4. Render the data in a ListView
5. Add forms for creating and editing notes
6. Connect the form to the backend through the service

This mirrors the Angular workflow of component + service + API integration.

17. Summary

If you know Angular, the fastest way to learn Flutter is to map concepts like this:

- Angular component → Flutter widget
- Angular template → widget tree
- Angular service → Dart service class
- Angular router → Navigator

- Angular state → StatefulWidget or state management package

Once that mapping is clear, Flutter becomes much easier to understand, especially when working with a Spring Boot backend.